

Отчет по лабораторной работе № 13 по курсу “Фундаментальная информатика”

Студент группы М80-107Б-20 Сысоев Максим Алексеевич, № по списку 23

Контакты e-mail, telegram, skype
masysoev@mai.education

Работа выполнена: «21 » ноября 2020г.

Преподаватель: каф. 806 Найденов Иван Евгеньевич

Отчет сдан <> ____ 20 ____ г., итоговая оценка ____

Подпись преподавателя

1 Тема: Множества

2 Цель работы: Решить задачу на языке Си при помощи реализации математической абстракции множества

3 Задание (вариант № 7): На вход подаётся произвольный набор английских слов, разделённых пробелами, запятыми, знаками табуляции и границами строк. Букву 'y' для простоты считать согласной (гласными или согласными бывают звуки, но не буквы). Необходимо проверить выполнение следующего условия: есть ли слова, начинающиеся и заканчивающиеся гласными? При решении задачи необходимо реализовать математическую абстракцию множества (сам тип, полное/пустое множество, пересечение, симметрическая разность и т.д.). Для реализации множества необходимо использовать битовые операции, но при этом запрещается нарушать принципы абстракции и инкапсуляции: пользовательский код должен зависеть только от интерфейса, но не от его реализации. При имплементации множеств можно считать, что все они имеют мощность не больше 32. Помимо обычных заголовочных файлов разрешается использовать ctype.h, inttypes.h и stdbool.h. Код функции main должен зависеть только от интерфейса АТД множество.

4 Оборудование (Студента):

Процессор AMD Ryzen 3 3200U @ 2.60 GHz с ОП 8 Гб, НМД 256 Гб. Монитор 1920x1080

5 Программное обеспечение (Студента):

Операционная система семейства: linux, наименование: ubuntu версия 14.1.0 интерпретатор команд: bash версия 4.4.19.

Система программирования -- версия --

Редактор текстов: emacs

Утилиты операционной системы --

Прикладные системы и программы: VS 2019

Местонахождение и имена файлов программ и данных на домашнем компьютере--

6. Идея, метод, алгоритм решения задачи (в формах: словесной, псевдокода, графической [блок-схема, диаграмма, рисунок, таблица] или формальные спецификации с пред- и постусловиями)

Для простоты я использовал знания, полученные в 11 лабораторной работе - использовал конечные автоматы.

Если первая буква - гласная, то ставим true(1) в переменную first. Затем переходим в конец слова и смотрим на последнюю букву. Если гласная - выводим Yes и завершаем программу.

Символы представляем в виде множества. Все гласные буквы = 1065233

Входной символ представляем в виде числа(логический сдвиг единицы на с - 'а' или с - 'А') и выполняем поразрядную конъюнкцию с числом 1065233. Если 0 - то, это не гласная буква, иначе - гласная.

7. Сценарий выполнения работы [план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию].

Тесты для программы:

Входные данные	Выходные данные	Описание тестирующего случая
amaretto	Yes	Классический случай
affectus sunt non levia	No	Много слов
Help please aee	Yes	Несколько пробелов между словами
AvA,YeaH	Yes	Заглавные буквы
Python or C++?	No	Программа не должна такое обрабатывать

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#include<stdio.h>

typedef enum {
    S0, S1, S2
} State;

int is_separator(char c)
{
    if (c == ' ' || c == ',' || c == '\n' || c == 9) {
        return 1;
    }
    return 0;
}

int is_small(char c)
{
    if (c >= 'a' && c <= 'z') {
        return 1;
    }
    return 0;
}

int main(void)
{
    int vowelLetters = 1065233;
    char c;
    int prew = 0;
    int first = 0;
    int last = 0;
    int sep = 0;
    State state = S0;
    unsigned int set = 0;

    while ((c = getchar()) != EOF) {
        if (!is_separator(c)) {
            if (is_small(c)) {
                set = 1u << (c - 'a');
            } else {
                set = 1u << (c - 'A');
            }
        }
        switch (state) {
```

```

case S0:
    if (sep) {
        if (last && first) {
            printf("Yes\n");
            return 0;
        } else {
            last = 0;
            first = 0;
            sep = 0;
            prew = 0;
        }
    }
    if (vowelLetters & set) {
        first = 1;
    }
    state = S1;
    break;
case S1:
    if (is_separator(c)) {
        if (vowelLetters & prew) {
            last = 1;
        }
        sep = 1;
        state = S2;
    } else {
        state = S1;
    }
    break;
case S2:
    if (is_separator(c)) {
        state = S2;
    } else {
        if (last && first) {
            printf("Yes\n");
            return 0;
        } else {
            last = 0;
            first = 0;
            sep = 0;
            prew = 0;
        }
    }
    if (vowelLetters & set) {

```

```
        first = 1;
    }
    state = S1;
}

}
prew = set;
if (last && first) {
    printf("Yes\n");
    return 0;
}
}
printf("No\n");
return 0;
}
```

9. Дневник отладки должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание
1	дом	21.11. .20	17:00	Неправильный ответ	Добавить обработку заглавных букв	

10. Замечания автора по существу работы

11. Выводы

Неплохая лабораторная работа - довольно простая. Думаю, что мат. абстракция множества будет полезным инструментом в будущем, ибо такой способ работы занимает меньше памяти (Если сравнивать с массивами, к примеру). Единственное, чему научился - работать с символами в таком ключе. В целом, ок.

Недочёты при выполнении задания могут быть устранены следующим образом:

Подпись студента
